

TU 257 Information Systems

2024-25

Assignment 2: Case Study Report – Airline Booking System

Student name: Eamonn Kelly

Student ID: D24127620

Date: 22nd Dec 2024

Part A – Conceptual Design

The following is the conceptual design for Part A of the assignment. I have chosen to do Airline booking system.

Design Description and Requirements

Some key requirements for an online airline booking database:

- **Flight Information:**
 - Store detailed information about flights, including flight number, departure and arrival locations, aircraft type, and seat capacity.
 - Maintain historical data on past flights for reporting and analysis.
- **Customer Information:**
 - Capture and store customer details, including names, contact information, and payment preferences.
- **Booking Records:**
 - Record details of each booking, linking customers to specific flights.
- **Payment Processing:**
 - Store payment details securely

Assumptions:

The following is a full list of assumption

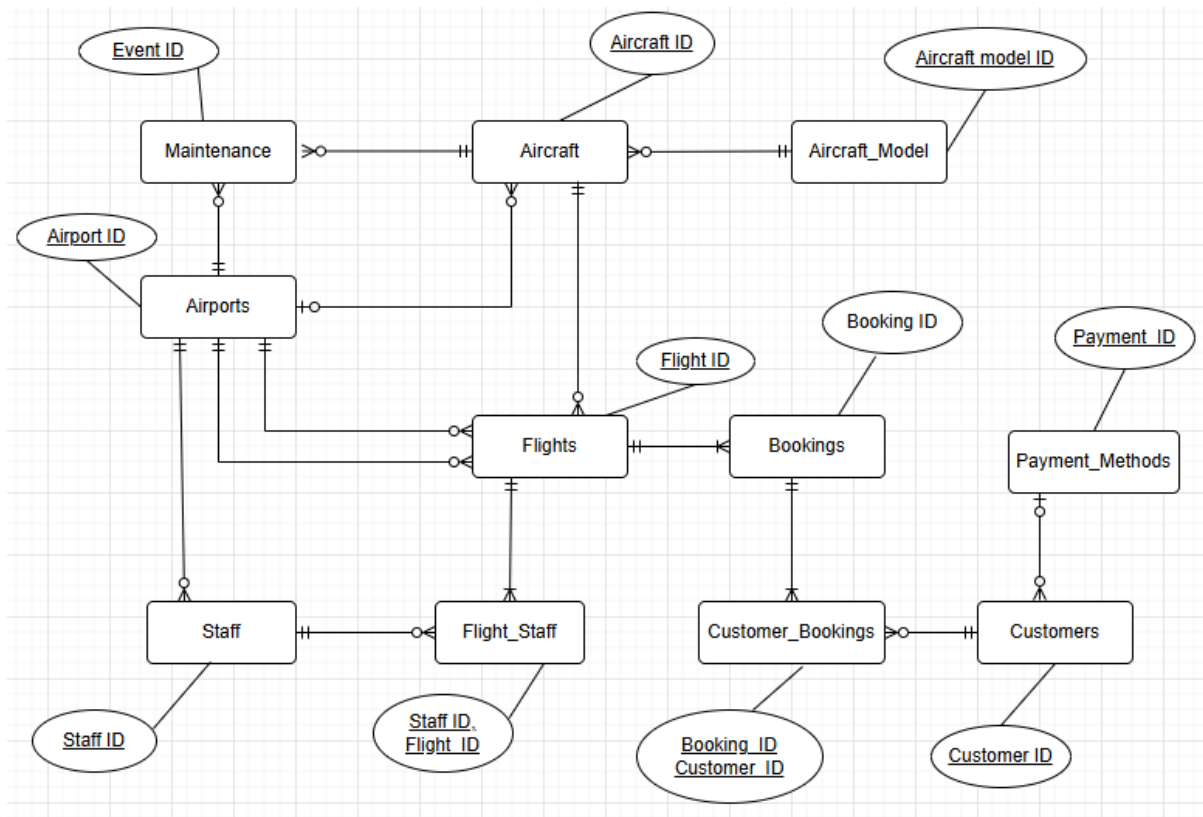
- An **Aircraft** may have **0** flights associated to it or it can have **many Flights**.
- Each **Flight** must have **1** and only **1 Aircraft** allocated to it.
- An **Aircraft Model** can have **0** or **many Aircraft** associated with it.
- An **Aircraft** must have **1** and only **1 Aircraft Model** associated with it.

- Each **Maintenance** event must have **1** and only **1 Aircraft** associated to it.
- Each **Aircraft** can have **0** or **many Maintenance** events.
- Each **Maintenance** event must have **1** and only **1 Airport** assigned to it. This is the location where the maintenance takes place. Aircraft maintenance events always take place in an airport.
- Each **Airport** can have **0** or **many Maintenance** events. Typically, maintenance events will take place at designated locations where the skills, tools, services and staff are available to perform the.
- Each **Airport** can have **0** or **many Aircraft** associated with it.
- Each **Aircraft** can have **0** or **1** only **Airport** associated with it. This is to allow us effectively know where the aircraft is currently located. If there is no airport listed the aircraft is currently in mid-flight.
- Each **Airport** can have **0** or **many Staff** associated with it.
- Each **Staff** member must have **1** and only **1 Airport** associated with them. This is the working location of the staff member and is required for contractual purposes. This is true even if the staff are flight staff and spend their working day in the air or travelling to different airports. Within the airport are office, aircraft, runway, warehouse, maintenance depot but this level of detail is not deemed required or essential to track. Location can be calculated or inferred by job title.
- Each **Aircraft** can have **0** or **many Flights** allocated to it.
- Each **Flight** must have **1** and only **1 Aircraft** associated with it, even if there is a stopover or transfer on a flight. Any second or continuing flight would be a separate flight and be treated as such.
- Each **Flight** has 1 starting(departure) airport and also has 1 ending(arrival) airport. Hence, we have two relationships between Flights and Airports to represent this.
 - Each **Flight** must have **1** and only **1 departure airport**
 - Each **Flight** must have **1** and only **1 arrival airport**
 - Each **Airport** can have **0** or **many departure flights**
 - Each **Airport** can have **0** or **many arrival flights**
- Each **Staff** member can have **0** or **many Flight_Staff** instances associated with them. The **Flight_Staff** entity is a bridging entity which represents staff allocation to flights, effectively a roster for the staff who are allocated to different flights. It has an *identifying* relationship between itself and **Staff** and **Flights**, it cannot exist without these two entities.
- Each **Flight** must have **1** or **many Flight_Staff** allocated to it. Each aircraft may have minimum requirements for staff to be onboard before it can safely fly as per regulations. These staff numbers can be calculated from the table data and to ensure flights can take place as per regulations and appropriately skilled staff can be allocated to flights in this entity.
- Each **Flight** must have **1** booking by may have **many Bookings**
- Each **Booking** must have **1** and only **1 Flight** associated with it.
- The **Customer_Bookings** entity is a bridging entity to allow multiple customers be associated with the same Booking. This facilitates Group bookings for the airline booking system. It has an *identifying* relationship between itself and **Bookings** and **Customers**, it cannot exist without these two entities. Group Booking are limited to 10 persons max, just so it is not open ended from a security, systems and aircraft capacity point of view.
- Each **Booking** must have **1** and but could have **many Customer_Bookings** associated with it.
- Each **Customer** can have **0** or may have **many Customer_Bookings** associated with them. A customer only has a relationship with a flight through a booking.

- Each **Customer** can have **0** but may have **1** **Payment_Method** associated with them.
- Each **Payment_Method** can have **0** but may have **many** **Customers** associated with it.
- Airport ID > is a 3 or 4 character text based code. It is an industry standard identifier. Hence, we use a VARCHAR data type.
- Available seats it a calculation on stored data. It is based off number of *seat_capacity* attribute in **Aircraft** and *seat_number* allocations in **Bookings**. The remainder being the available seats. It is not stored in the database. It is also a value that could change quite regularly. Including available seats would also create transitive dependencies across tables which we would need to address and given we have a max number of entities allowed, it is best not to store this but calculate at run time.
- No account of Ghost flights, where no customers are on board is accounted for. Assuming empty flights (ghost flights) flying with no one on board due to contractual obligations does not occur in the airline.
- Price >> is recorded in Bookings so users can refer to it and have it available historically. The value in real time though may change periodically based on updated calculations and algorithms.
- Distance between airports (dep and arrival) will have impact on fuel required for flight. This could be calculable data presented to the pilots.
- FLIGHTS >> Departure and arrival airports cannot be the same airport, dep and arrival must be different
- Bookings > seat_number attribute. It can be null as seat number not chosen/allocated until user checks in, so can make booking and leave as null until check in time. It can be used in reports to identify where someone is sitting, or available seats through calculated query.
- Bookings > price attribute >>> we assume all pricing is done in euros, as such we will not include any currency conversion or support for other currencies
- No discounting data is stored. Must be calculated if needed.

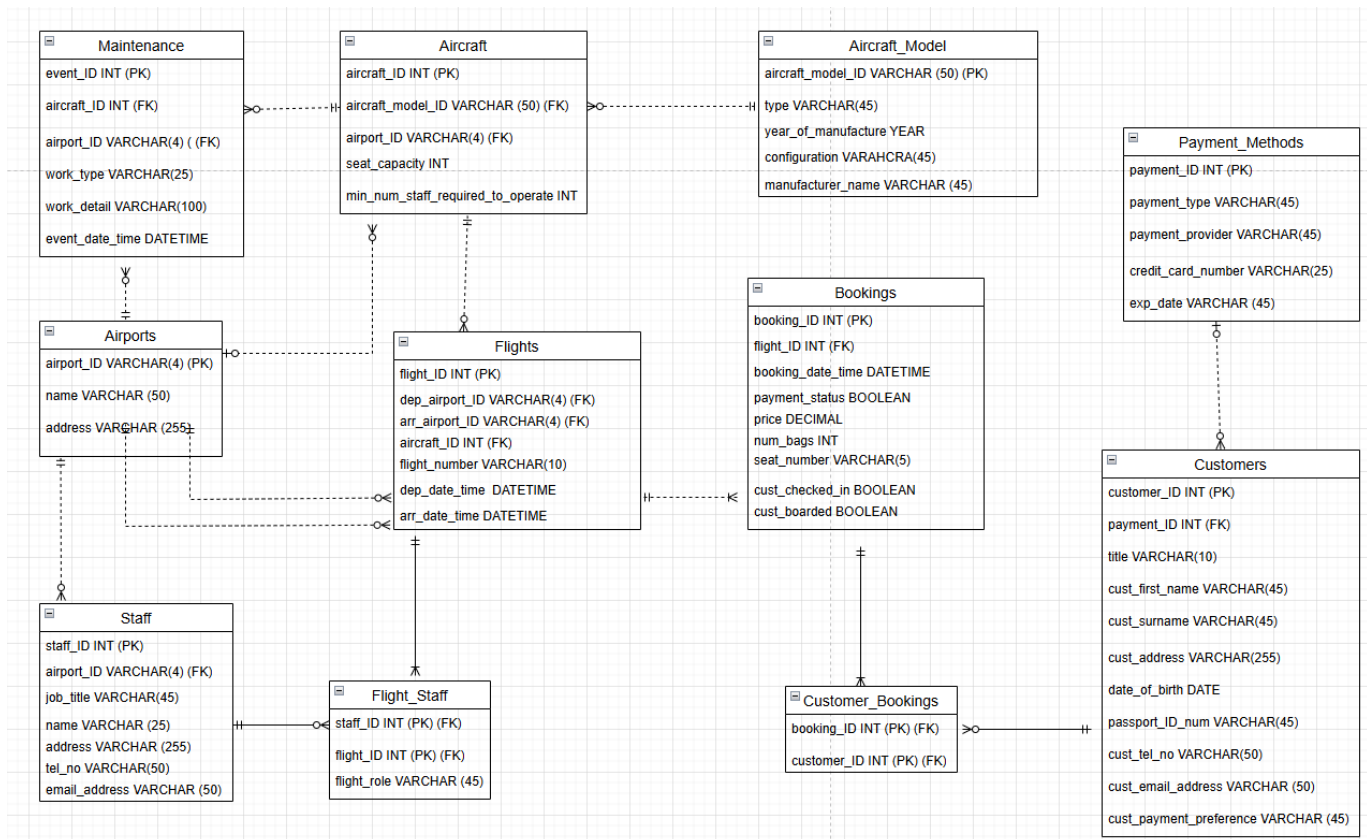
Conceptual design ER diagram including main entities and cardinality

Below is the rough ER diagram containing entities, primary keys cardinality.



Conceptual design Final ER diagram full list of attributes and PKs/FKs

Below is out fully attributable ER diagram containing entities, primary keys, foreign keys, optionality and cardinality, relationships, full list of attributes and the associated data types.



Part B – Conceptual Design

Screenshots of typical data

The following is a screenshot of typical personal customer data that a customer might be required to provide.

Personal Customer Data required - Sample record		
title	Mr/Mrs/Ms	< one of 3 options - text>
Customer Name	<first name>	text
Customer surname	<surname>	text
date of birth	<YYYY-MM-DD>	date - must be in this format for MYSQL
address	<address>	text and number
tel	<+0044 1234567 890 >	is a number but could have leading zeros, plus sign, spaces, so we will use varchar
email address	< abc@abc.com	text and and no standard characters
passport/ID number	<A12345678>	letters and numbers - varchar

The following is a screenshot of typical data that a customer might be required to provide to make a flight booking.

Flight information a customer may require - Sample record		
Departure airport	City Name	typed name of city VARCHAR
Arrival airport	City Name	typed name of city VARCHAR
Departure date	Date	DATE data type
Arrival date	Date	DATE data type
Departure time	Time	DATE TIME data type
Arrival Time	Time	DATE TIME data type
number of individuals travelling	between 1 - 10 (limit may vary)	(limit may vary) - INT
age of individuals travelling	1 to 100	maybe place a limit on it... 100
Number of Bags	1 to 10	max limit of 1 to 10 - INT

Normalization process

Our Entities and attributes are already in 3NF. We will outline why that is the case now. Below are our tables and attributes.

Maintenance(event_ID (PK), aircraft_ID (FK), airport_ID (FK), work_type, work_detail, event_date_time)

Aircraft(aircraft_ID (PK), aircraft_model_ID (FK), airport_ID (FK), seat_capacity, min_num_staff_required_to_operate)

Aircraft_Model(aircraft_model_ID (PK), type, year_of_manufacture, configuration, manufacturer_name)

Airports(airport_ID (PK), name , address)

Flights(flight_ID (PK), dep_airport_ID (FK), arr_airport_ID (FK), aircraft_ID(FK), flight_number, dep_date_time , arr_date_time)

Bookings(booking_ID (PK), flight_ID (FK), booking_date_time, payment_status, price, num_bags, seat_number, cust_checked_in, cust_boarded)

Staff(staff_ID (PK), airport_ID (FK), job_title, name, address, tel_no, email_address)

Flight_Staff(staff_ID (FK), flight_ID (FK), flight_role)

Customers(customer_ID (PK), payment_ID (FK), title, cust_first_name, cust_surname, cust_address, date_of_birth, passport_ID_num, cust_tel_no, cust_email_address), cust_payment_preference)

Customer_Bookings(booking_ID (FK), customer_ID (FK))

Payment_Methods(payment_ID (PK), payment_type, payment_provider, credit_card_number, exp_date)

1NF

The tables are in 1NF already and no changes are needed as all tables have primary keys and all attributes have atomic values i.e. there are no repeating groups, or attributes containing multiple values.

2NF

The tables are already in 2NF and no changes are needed. There are two tables with composite Primary keys which could contain partial dependencies, listed below. No partial dependencies exist in these two tables however so as such we are at 2NF.

Flight_Staff(staff_ID (PK) (FK), flight_ID (PK) (FK), flight_role)

Customer_Bookings(booking_ID (PK) (FK), customer_ID (PK) (FK))

3NF

The tables are already in 3NF and no changes are needed. There are no transitive dependencies across any of the attributes in each table. Each attribute is wholly dependent on the primary key, not dependent on any other non-primary attribute.

As such no changes are needed as part of the normalization process and the normalization process is complete, as we are at 3NF.

Physical design

Creation of schema

We created the tables initially manually and subsequently using sql cmds. We include an example of one such statement for the Bookings table.

```
CREATE TABLE IF NOT EXISTS Bookings (  
    booking_ID INT PRIMARY KEY,  
    flight_ID INT,  
    booking_date_time DATETIME,  
    payment_status BOOLEAN,  
    price DECIMAL(10, 2),  
    num_bags INT  
    seat_number VARCHAR(5),  
    cust_checked_in BOOLEAN,  
    cust_boarded BOOLEAN,  
    FOREIGN KEY (flight_ID) REFERENCES Flights(flight_ID)  
);
```

By default, this Included FK constraints in tables and ensured data referential integrity in the sample data, making sure data values had to exist in the foreign key data.

We then generated some sample data using Mockaroo and ChatGPT and added that data to the tables by a mixture of sql commands and csv imports in the **Table Data Import Wizard**. An example of the sql insert statement we used is included below. Some manual manipulation of the data was required to ensure the referential integrity was maintained.

```
INSERT INTO staff (staff_ID, airport_ID, job_title, name, address, tel_no, email_address)
VALUES
(1, 'JFK', 'Technician', 'Adam Turner', '503 Kennedy Camp Suite 808 New Andreaport, MO 49781',
1219764428, 'mhernandez@gmail.com'),
(2, 'LHR', 'Technician', 'Carol Anderson', '6897 Villanueva Ridges Apt. 038Ashleyburgh, MS 65240',
3606746064, 'watsonholly@hotmail.com'),
(3, 'LAX', 'Technician', 'Jessica Washington', 'Unit 0285 Box 6394 DPO AP 58941', 9774656145,
'gregorypaul@jenkins.com'),
(4, 'DXB', 'Cabin Crew', 'Samuel Eaton', '4488 Karen Walks Mariaport, NV 68824', 8320139695,
'scott98@robinson-browning.com'),
(5, 'JFK', 'Technician', 'Peter Nichols', '06886 Tiffany Station Port Nancymouth, SC 63879',
1137820076, 'buckamanda@gmail.com'),
```

SQL Queries

The following is the list of queries I ran in the video.

- 1) View all data in the named table

```
select * from airlinedb_ek_v2.flights
select * from airlinedb_ek_v2.bookings
select * from airlinedb_ek_v2.customers
```
- 2) How many pilots do we have?

```
select COUNT(*) from airlinedb_ek_v2.staff where job_title='pilot'
```
- 3) How many Boeing 777 aircraft type do we have?

```
select COUNT(*) from airlinedb_ek_v2.aircraft where aircraft_model_ID='B777'
```
- 4) View airport names where the name starts with the letter 'd'

```
select * from airlinedb_ek_v2.airports where name like 'd%'
```
- 5) Insert a new airport record

```
INSERT INTO airlinedb_ek_v2.airports VALUES
('DUB', 'Dublin Airport', 'airport road, swords, Dublin 17');
```
- 6) Update a record

```
Update airlinedb_ek_v2.airports
set address='52 airport road, swords, Dublin 21'
where airport_ID='DUB'
```
- 7) Delete a record

```
delete from airlinedb_ek_v2.airports where airport_ID='DUB'
```
- 8) How many customers with names containing the letter 'm', not case sensitive, in their name?

```
SELECT * from airlinedb_ek_v2.customers where cust_first_name like 'm%'
```
- 9) How many customers with names containing the letter 'd' in their name and who have a 'gmail' email address?

```
SELECT * from airlinedb_ek_v2.customers where cust_first_name like '%d%'
and cust_email_address like '%gmail%';
```

10) What is the max price of flights for bookings?

```
SELECT MAX(airlinedb_ek_v2.bookings.price) FROM airlinedb_ek_v2.bookings
```

11) What is the min price of flights for bookings?

```
SELECT MIN(airlinedb_ek_v2.bookings.price) AS Min_Flight_Price FROM  
airlinedb_ek_v2.bookings
```

12) What is the average price of flights for bookings?

```
SELECT avg(airlinedb_ek_v2.bookings.price) AS Avg_Flight_Price FROM  
airlinedb_ek_v2.bookings
```

13) How many flights go to Heathrow from 2024?

```
SELECT count(*) from airlinedb_ek_v2.flights where arr_airport_ID='LHR'
```

14) How many flights go to Heathrow during December 2024?

```
SELECT count(*) from airlinedb_ek_v2.flights where arr_airport_ID='LHR'  
and dep_date_time >= '2024-12-01'  
and dep_date_time <= '2024-12-31'
```

15) What aircraft has the highest number of maintenance events and in which airport does it occur?

```
SELECT aircraft_ID, count(*) as num_of_maintenance_events from  
airlinedb_ek_v2.maintenance  
group by aircraft_ID  
order by num_of_maintenance_events DESC
```

16) Customers with surname starting with letter R using REGEX

```
SELECT * from airlinedb_ek_v2.customers where cust_surname REGEXP '^R';
```

17) What type of aircraft has the highest number of maintenance events? Using an INNER JOIN

```
SELECT airlinedb_ek_v2.aircraft.aircraft_model_ID, count(*) as  
num_of_maintenance_events  
from airlinedb_ek_v2.maintenance  
join airlinedb_ek_v2.aircraft ON  
airlinedb_ek_v2.maintenance.aircraft_ID=airlinedb_ek_v2.aircraft.aircraft_ID  
group by airlinedb_ek_v2.aircraft.aircraft_model_ID  
order by num_of_maintenance_events DESC
```

18) Average price of flight to LHR – using s Sub Query

```
select AVG(airlinedb_ek_v2.bookings.price) AS avg_LHR_flight_price From  
airlinedb_ek_v2.bookings
```

```
where flight_ID IN(Select flight_ID from airlinedb_ek_v2.flights where
airlinedb_ek_v2.flights.arr_airport_ID = 'LHR')
```

19) Average price of flight to LHR – Sub Query

```
select AVG(airlinedb_ek_v2.bookings.price) AS avg_LHR_flight_price From
airlinedb_ek_v2.bookings where flight_ID IN(Select flight_ID from
airlinedb_ek_v2.flights where airlinedb_ek_v2.flights.arr_airport_ID = 'LHR')
```

VIEW

1) 'engineers' - The details of staff who are engineers

```
CREATE VIEW `ground_crew_2` AS
Select * FROM `staff`
WHERE (`staff`.`job_title` NOT IN ('pilot' , 'cabin crew', 'ground staff'))
```

2) 'ground_staff' - The details of staff who are not flight crew i.e. ground crew

```
CREATE VIEW `ground_crew_2` AS
Select * FROM `staff`
WHERE (`staff`.`job_title` NOT IN ('pilot' , 'cabin crew', 'ground staff'))
```

3) 'lax_flights' - The number of flights to LAX Airport and the passenger names travelling on those flights. Uses a JOIN of flights to bookings and bookings to customer

```
SELECT
    flights.flight_ID AS flight_ID,
    customers.cust_first_name AS cust_first_name,
    customers.cust_surname AS cust_surname
FROM
    (((flights
    JOIN bookings ON ((flights.flight_ID = bookings.flight_ID)))
    JOIN customer_bookings ON ((bookings.booking_ID = customer_bookings.booking_ID)))
    JOIN customers ON ((customer_bookings.customer_ID = customers.customer_ID)))
WHERE
    (flights.arr_airport_ID = 'lax')
```

4) 'lhr_flights'

The number of flights to LHR and the passenger names travelling on those flights. Uses a JOIN of flights to bookings and bookings to customer

```
SELECT
    flights.flight_ID AS flight_ID,
    customers.cust_first_name AS cust_first_name,
```

```
customers.cust_surname AS cust_surname
FROM
  (((flights
    JOIN bookings ON ((flights.flight_ID = bookings.flight_ID)))
    JOIN customer_bookings ON ((bookings.booking_ID = customer_bookings.booking_ID)))
    JOIN customers ON ((customer_bookings.customer_ID = customers.customer_ID)))
WHERE
  (flights.arr_airport_ID = 'lhr')
```

Index

I chose to add 4 non-unique indices

- 1) flights.flight_number
- 2) flights.arr_airport_ID
- 3) flights.arr_date_time
- 4) bookings.price (Non unique key - could have flights with same price and this allows that)

Indices are useful where they are queried a lot in WHERE statement and JOINS. Flight destination, departure times and price would be such data and as such we felt these 3 were good candidates to try improve efficiency and optimize the database for quicker response times. We cannot see an impact ourselves in our database however as the quantity of the data is small hence the data is returned quickly before and after adding the indices anyway.